

Spark Debugging + Performance Cheatsheet

Triage loops, Spark UI evidence, tuning levers, commands, and PySpark snippets

A2 FIELD GUIDE

Debugging Triage Loop

- Name the symptom: slow stage, failed task, driver OOM, executor OOM, wrong row count, empty output.
- Find the first failing or dominant stage in the Spark UI.
- Compare task duration, input size, shuffle read/write, spill, and GC time.
- Open EXPLAIN FORMATTED and locate Scan, Filter, Exchange, Join, Aggregate, Sort, Window.
- Change one lever, rerun on a representative sample, then compare evidence.

Spark UI Tabs To Read

Tab	Use it for
Jobs	Actions triggered, repeated lineage, failed jobs.
Stages	Task counts, skew, shuffle, spill, stragglers.
SQL	Physical operators, joins, exchanges, AQE changes.
Storage	Cached data size, memory vs disk, replication.
Executors	Lost executors, GC, memory, task failures.

Plan Clues

- Exchange: Spark is redistributing data. Expect network, disk, and skew risk.
- BroadcastExchange: one side is sent to executors. Great when small; dangerous when mis-sized.
- SortMergeJoin: both sides often sorted and shuffled by join key.
- HashAggregate: usually partial aggregate before shuffle, final aggregate after shuffle.
- AdaptiveSparkPlan: AQE may change joins, coalesce partitions, or handle skew at runtime.

Symptom -> Inspect -> Fix

Symptom	Inspect	Likely fixes
Driver OOM	collect, toPandas, huge plan, tiny partition metadata, driver logs.	Keep output distributed, limit/sample, write to storage, simplify lineage, reduce tiny files.
Executor OOM	Executor logs, task input, spill, join side size, skewed keys.	Reduce partition size, fix skew, broadcast only small tables, persist with care, raise memory after evidence.
One slow task	Task duration histogram, shuffle read by task, spill, key frequencies.	Salt hot keys, pre-aggregate, filter invalid heavy keys, enable/tune AQE skew handling.
Few cores active	Stage task count, input splits, coalesce/repartition, shuffle partitions.	Increase partitions, avoid coalesce(1), repartition for balance, compact small files upstream.
Slow join	Join strategy, Exchange count, table sizes, null/hot keys.	Broadcast small side, project/filter first, repartition by key, handle skew, validate join cardinality.
Repeated work	Multiple jobs scanning same source or repeating same shuffle.	Persist reused expensive DataFrames, materialize once, remove unnecessary count/show actions.

Performance Levers

- Partition size: enough tasks for parallelism, not so many that scheduling dominates.
- Shuffle partitions: tune `spark.sql.shuffle.partitions`; AQE can coalesce after shuffle.
- Join strategy: broadcast small side; avoid shuffling two large skewed inputs blindly.
- File layout: compact tiny files; avoid single huge files and many tiny output files.
- Cache only reused expensive results; call an action to materialize; unpersist when done.
- Push down filters and select columns early.

Shell + Spark Commands

common commands

```
pyspark
spark-submit app.py
spark-submit --master local[*] app.py
spark-submit --conf k=v app.py
spark-submit --packages group:artifact:ver app.py
spark-sql
spark-shell
df.explain('formatted')
spark.catalog.listTables()
spark.catalog.clearCache()
spark.stop()
docker compose up -d
docker compose ps
docker compose logs spark
docker compose exec spark bash
docker compose down
```

PySpark Snippets

dataframe API

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window
df.printSchema()
df.dtypes
df.select('a','b').where(F.col('a') > 0)
df.groupBy('key').agg(F.count('*'), F.avg('x'))
w = Window.partitionBy('key').orderBy(F.desc('x'))
df.withColumn('rn', F.row_number().over(w))
df.join(dim, 'id', 'left')
F.broadcast(dim)
df.repartition(200, 'key')
df.coalesce(20)
df.persist(); df.count()
df.unpersist()
df.write.mode('overwrite').parquet(path)
spark.read.schema(schema).csv(path, header=True)
```

Config Quick Refs

configs + checks

```
spark.conf.get('spark.sql.shuffle.partitions')
spark.conf.set('spark.sql.shuffle.partitions','200')
spark.conf.set('spark.sql.adaptive.enabled','true')
spark.conf.set('spark.sql.autoBroadcastJoinThreshold','50MB')
spark.conf.set('spark.sql.files.maxPartitionBytes','128MB')

# inspect shape safely
df.rdd.getNumPartitions()
df.limit(20).show(truncate=False)
df.sample(0.01).count()
df.where(F.col('key').isNull()).count()
df.select('key').groupBy('key').count()
    .orderBy(F.desc('count')).show()
```